

Table of contents

Data Type Sizes Across Architecture.....	4
Memory Segment Address Matching.....	6
Pointer and Memory Allocation.....	8
Knowledge Check.....	9
GitHub Link.....	9

List of figures

Figure 1: Program to Observe Data Type Sizes.....	4
Figure 2: Making the file executable.....	4
Table 1: Comparing the Output.....	5
Figure 3: Program to Observe Variable Addresses.....	6
Figure 4: Commands to Make File Executable.....	6
Figure 5: Difference between Stack and Heap.....	7
Figure 6: Correct Pointer and Heap Usage.....	8
Figure 7: Terminal Commands.....	8

List of Tables

Table 1: Comparing the Output.....	5
------------------------------------	---

Data Type Sizes Across Architecture

```
#include <stdio.h>

int main() {
    printf("Data Type Sizes on This System \n");

    printf("char: %zu bytes\n", sizeof(char));
    printf("int: %zu bytes\n", sizeof(int));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));
    printf("long: %zu bytes\n", sizeof(long));
    printf("pointer: %zu bytes\n", sizeof(int*));
    printf("long long: %zu bytes\n", sizeof(long long));
    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));

    return 0;
}
```

Figure 1: Program to Observe Data Type Sizes

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week4/lab4$ nano datatype.c
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week4/lab4$ gcc datatype.c -o datatype
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week4/lab4$ ./datatype
Data Type Sizes on This System
char: 1 bytes
int: 4 bytes
float: 4 bytes
double: 8 bytes
long: 8 bytes
pointer: 8 bytes
long long: 8 bytes
unsigned int: 4 bytes
```

Figure 2: Making the file executable

Laboratory Exercises:

Comparing output with Table data

Data Type	64-bit expected	My Output
char	1	1
int	4	4
float	4	4
double	8	8
long	8	8
pointer	8	8
long long	8	8
unsigned int	4	4

Table 1: Comparing the Output

Q: Why might sizeof(long) differ between 32-bit and 64-bit systems?

A: long datatype can have different size between 32-bit and 64-bit because they handle memory address differently.

Summary

This program used sizeof() to find how much memory different data types use. We observed that different datatypes require different amounts of memory and it can also differ on the basis of memory address.

Memory Segment Address Matching

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 150; //Data segment
int bss_var; //BSS segment

int main() {
    int local_var = 30; //Stack
    static int local_static = 20; //Data segment

    int *heap_var = (int *)malloc(sizeof(int)); //Heap
    *heap_var = 500;

    printf("Variable addresses\n");
    printf("global_var: %p (Data)\n", (void*)&global_var);
    printf("bss_var: %p (BSS)\n", (void*)&bss_var);
    printf("local_var: %p (Stack)\n", (void*)&local_var);
    printf("local_static: %p (Data)\n", (void*)&local_static);
    printf("heap_var ptr: %p (Stack - pointer)\n", (void*)&heap_var);
    printf("*heap_var: %p (Heap - data)\n", (void*)heap_var);
    free(heap_var);
    return 0;
}
```

Figure 3: Program to Observe Variable Addresses

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week4/lab4$ nano datatype.c
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week4/lab4$ nano address.c
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week4/lab4$ gcc address.c -o address
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week4/lab4$ ./address
Variable addresses
global_var: 0xb3bbf2100010 (Data)
bss_var: 0xb3bbf210001c (BSS)
local_var: 0xffffd8212a2c (Stack)
local_static: 0xb3bbf2100014 (Data)
heap_var ptr: 0xffffd8212a30 (Stack - pointer)
*heap_var: 0xb3bc3003e010 (Heap - data)
```

Figure 4: Commands to Make File Executable

Laboratory Exercises:

Q: Identify which segment has the highest addresses.

A: Stack has the highest address (0xffffd8212a2c).

Q: Identify which segment has the lowest addresses.

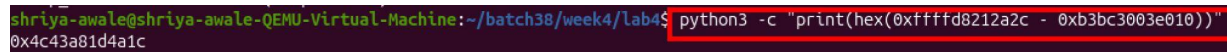
A: Heap has the lowest address (0xb3bc3003e010).

Q: Calculate the address difference between stack and heap.

A:

Stack: 0xffffd8212a2c

Heap: 0xb3bc3003e010

A terminal window with a dark background. The prompt is 'shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week4/Lab4\$'. The command entered is 'python3 -c "print(hex(0xffffd8212a2c - 0xb3bc3003e010))"'. The output is '0x4c43a81d4a1c'. The command and output are highlighted with a red box.

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/week4/Lab4$ python3 -c "print(hex(0xffffd8212a2c - 0xb3bc3003e010))"
0x4c43a81d4a1c
```

Figure 5: Difference between Stack and Heap

Summary

This program printed the addresses of variables stored in different memory segments. We observed that each type of variable is stored in a different memory area. Further we observed that Stack has the highest address whereas Heap has the lowest address, and their difference was also calculated.

Pointer and Memory Allocation

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *forheap;
    forheap = (int *)malloc(sizeof(int));
    *forheap = 30;

    printf("Address of pointer (STACK): %p\n", (void*)&forheap);
    printf("Address of data (HEAP): %p\n", (void*)forheap);
    printf("Value stored: %d\n", *forheap);

    return 0;
}
```

Figure 6: Correct Pointer and Heap Usage

```
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab4$ nano pointer.c
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab4$ gcc pointer.c -o pointer
shriya-awale@shriya-awale-QEMU-Virtual-Machine:~/batch38/lab4$ ./pointer
Address of pointer (STACK): 0xffffdb9f0e20
Address of data (HEAP): 0xc87902446010
Value stored: 30
```

Figure 7: Terminal Commands

Summary

This program created a space in memory to store the value 30. It showed that the pointer and the actual value are kept in two different places in memory.

Knowledge Check

1) What is a data type in the context of programming and operating systems?

A: It tells the system what kind of data is stored by a variable and how much memory it needs.

2) Name the five main memory segments in a process memory layout.

A: Stack, BSS, Data, Text, and Heap

3) What is the difference between the Data segment and the BSS segment?

A: Data segment stores global variables which are initialized, whereas BSS segment stores global variables which are not initialized.

4) Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

A: Since, stack grows downwards and heap grows upwards, they require space. To make sure they don't overlap they are stored at certain addresses.

5) What happens to stack memory when a function returns? Who reclaims it?

A: When a function finishes, its temporary memory is automatically cleared. The OS manages it.

6) How does the OS know which memory belongs to which process?

A: The OS keeps a separate virtual address and memory information for each process.

GitHub Link: <https://github.com/shriya07-debug/Operating-Systems-Security-and-Networks.git>